

# 1 WarehouseIQ — Flow Command: Comprehensive Project Specification, Architecture, and Presentation Master Manual

**Author:** Manus AI

**Project Name:** smart-warehouse-operations

**Live Application URL:** <https://smartwarehouse-nwqedban.manus.space>

**GitHub Repository:** <https://github.com/saisudheernallabarika77-alt/Smart-Warehouse-Operations>

---

## 1.1 Section 1: Executive Summary & Hackathon Problem Statement

Fulfillment centers operate at extreme velocity, managing thousands of active SKUs, inbound replenishment containers, dynamic pick waves, and strict carrier dispatch cut-off times. Traditional warehouse management systems (WMS) function as passive ledgers—recording inventory movements after anomalies occur. **WarehouseIQ — Flow Command** revolutionizes this model by providing an executive decision-driven smart warehouse operations platform that predicts bottlenecks, resolves stockout disputes through intelligent split-allocation algorithms, and guides shift leads with real-time decision intelligence.

### 1.1.1 The Hackathon Problem

Warehouses handle multi-item orders simultaneously. Poor inventory visibility, incorrect stock allocation, delayed picking, misplaced items, and fulfillment bottlenecks result in delayed shipments, severe stockouts, and dissatisfied customers.

**The Competitive Twist:** An application should *not just display data*—it must help make decisions. For example, when an urgent order requires 10 units but only 7 are available while another lower-priority order requires 5 units, what should the system decide?

### 1.1.2 Our Decision-Driven Solution

Instead of simple CRUD tables, WarehouseIQ implements an active **Exception → Decision → Resolution** workflow:

1. **Inventory Radar & Anomaly Detection:** Real-time monitoring of SKU stock levels, identifying impending stockouts and damaged goods before fulfillment halts.
2. **Intelligent Allocation & Prioritization:** Automated ranking of order queues by SLA urgency, carrier cut-off times, and revenue impact. When stock is constrained, the system evaluates split-allocation options and prompts operators with 1-click approval actions.
3. **Shift-Specific Workspaces:** Complete operational isolation between Shift A (Morning Inbound), Shift B (Afternoon Peak Fulfillment), and Shift C (Night Replenishment). Changing shifts alters all command desk metrics, task queues, and dispatch schedules.
4. **AI Prediction Copilot & Chatbot:** A project-aware conversational assistant integrated directly into the command suite, powered by a secure server-side LLM proxy that answers operator questions instantly.

---

## 1.2 Section 2: Complete System Architecture & Code Structure

WarehouseIQ is built on a clean, modular full-stack architecture that separates domain types, shift datasets, UI components, backend routers, and automated tests.

### 1.2.1 Directory Tree & File Purpose

```
smart-warehouse-operations/
├─ client/
│   └─ src/
│       └─ components/          # Modular UI components
│           └─ AppShell.tsx     # Sidebar, persistent rail, shift selector, Sudheer
account panel
│           └─ CommandDesk.tsx  # High-level operational metrics and signal feeds
│           └─ OrderQueueWorkspace.tsx # Order prioritization, allocation, and
split-view
│           └─ Workspaces.tsx   # Inventory Radar, Pick & Pack, Dispatch,
Analytics, Order Drawer
│           └─ AiPredictionWorkspace.tsx # Project-aware AI chatbot and decision
intelligence
│       └─ data/
│           └─ warehouseData.ts # Shift-specific datasets (Shift A, B, C)
│       └─ types/
│           └─ warehouse.ts     # TypeScript domain models and interfaces
│       └─ lib/
│           └─ warehouseAssistant.ts # Deterministic knowledge layer and fallback
engine
│           └─ pages/
│               └─ Home.tsx      # Main dashboard orchestrator
├─ server/
│   └─ _core/                   # Framework plumbing, Vite bridge, LLM proxy helper
│   └─ routers/
│       └─ ai.ts                # Secure server-side tRPC AI procedure (`ai.ask`)
│       └─ routers.ts           # Root tRPC router registry
│       └─ ai.ask.test.ts       # Vitest unit regression test for AI procedure
├─ drizzle/                    # Database schema and migration definitions
├─ PROJECT_DOCUMENTATION.md    # Detailed technical README
└─ WarehouseIQ_Complete_Presentation_Guide.pdf # Master PDF guide
```

## 1.3 Section 3: Technology Stack & Tooling Breakdown

| Layer / Domain  | Technology / Library                              | Exact Purpose & Role in WarehouseIQ                                                                               |
|-----------------|---------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| Frontend UI     | React 19 & Tailwind CSS 4                         | Modern component rendering, responsive grid layouts, and custom design tokens.                                    |
| Design System   | shadcn/ui & Lucide Icons                          | Accessible UI primitives (dialogs, drawers, badges) and professional industrial iconography.                      |
| Client Routing  | Wouter 3.3                                        | Lightweight, hook-based client-side routing and URL state management.                                             |
| Full-Stack API  | tRPC 11 & Express 4                               | Type-safe RPC procedures connecting frontend clients to server-side business logic without boilerplate REST code. |
| Database & ORM  | MySQL 8 / TiDB & Drizzle ORM                      | Relational data persistence, schema definitions, and query helper abstractions.                                   |
| AI Integration  | Manus Built-in LLM Proxy (BUILT_IN_FORGE_API_KEY) | Server-side secure AI question answering without exposing credentials in client-side bundles.                     |
| Testing & Build | Vitest & TypeScript (tsc)                         | Automated unit testing (ai.ask.test.ts) and zero-error compilation checks.                                        |
| Version Control | Git & GitHub CLI (gh)                             | Continuous code synchronization to <code>saisudheernallabarika77-alt/Smart-Warehouse-Operations</code> .          |

## 1.4 Section 4: End-to-End Execution Workflow & Code Flow

### 1. Shift Selection & State Sync (AppShell.tsx & Home.tsx):

- User selects a shift (Shift A, B, or C) from the top-right menu or sidebar.

- The entire dashboard dynamically re-indexes its data: Command Desk metrics, order queue priorities, inventory radar levels, pick-and-pack wave assignments, and dispatch cut-off schedules.
2. **Order Fulfillment Lifecycle:**
- **Order Created → Priority Determined → Inventory Checked → Allocation Decision → Picking & Packing → Quality Check → Dispatch.**
  - When inventory is constrained, automated allocation recommendations offer split-shipment or backorder holds with 1-click approvals.
3. **Secure AI Copilot Chatbot Flow:**
- User types a question in the AI Prediction module (`AiPredictionWorkspace.tsx`).
  - Client sends prompt + active shift context to the server-side tRPC procedure `ai.ask`.
  - The server calls the Manus built-in LLM proxy using `BUILT_IN_FORGE_API_KEY` (securely kept server-side).
  - If offline, `warehouseAssistant.ts` instantly provides a deterministic fallback response.
- 

## 1.5 Section 5: Mentor Presentation Q&A Briefing

- **Q: Is this connected to a real physical warehouse?**  
A: No, it is a high-fidelity WMS simulation using robust mock datasets and shift-aware states that behave identically to an enterprise fulfillment center.
  - **Q: How do shifts differ if the data is mocked?**  
A: Each shift has a dedicated operational profile in `warehouseData.ts`. Changing shifts instantly transforms command desk metrics, active order queues, and AI recommendations.
  - **Q: Which API key was used, and how is it secured?**  
A: We used the Manus secure server-side built-in LLM proxy (`BUILT_IN_FORGE_API_KEY`) via a tRPC procedure (`ai.ask`). No API key is ever exposed in the browser.
- 

[1] WarehouseIQ Live Preview URL: <https://smartwarehouse-nwqedban.manus.space>

[2] GitHub Repository: <https://github.com/saisudheernallabarika77-alt/Smart-Warehouse-Operations>

[3] Technical Documentation: `PROJECT_DOCUMENTATION.md`